

Sanchay AWS + Terraform Session — Command Reference

Every command run during the Sanchay AWS/Terraform setup session, with a one-line explanation.

1. Terraform Basics

Check installed Terraform version

```
terraform -version
```

Download providers and set up the working directory

```
terraform init
```

Preview changes before applying them

```
terraform plan
```

Apply changes to real infrastructure

```
terraform apply
```

2. Terraform — Targeted Plan/Apply (single module)

Plan only the ecr module, with placeholder values for required vars

```
terraform plan -target=module.ecr -var-file=environments/dev/terraform.tfvars  
-var="container_image=placeholder" -var="database_url=placeholder"
```

Apply only the ecr module

```
terraform apply -target=module.ecr -var-file=environments/dev/terraform.tfvars  
-var="container_image=placeholder" -var="database_url=placeholder"
```

Full plan with real container image and database_url

```
terraform plan -var-file=environments/dev/terraform.tfvars \  
-var="container_image=<ecr-uri>:latest" \  
-var="database_url=$DATABASE_URL"
```

Full apply, using a dedicated state file per environment

```
terraform apply \  
-var-file=environments/prod/terraform.tfvars \  
-var="container_image=<account-id>.dkr.ecr.us-east-2.amazonaws.com/sanchay-prod-api:latest" \  
-var="database_url=$DATABASE_URL" \  
-state=environments/prod/terraform.tfstate
```

3. Terraform — Import Existing AWS Resources

Import a resource into Terraform state (Terraform 1.5+ import block, in a .tf file)

```
import {  
  to = module.networking.aws_vpc.main  
  id = "vpc-0abc123..."  
}
```

Generate .tf config automatically for resources listed in import blocks

```
terraform plan -generate-config-out=generated.tf
```

Older imperative import (works on any Terraform version)

```
terraform import aws_s3_bucket.existing your-existing-bucket-name
```

4. Terraform — Cleanup / Reset Local State

```
# Remove local state, lock file, and provider cache to start fresh
rm -rf .terraform .terraform.lock.hcl terraform.tfstate*

# Windows PowerShell equivalent
Remove-Item .terraform.lock.hcl, terraform.tfstate*, .terraform -Recurse -Force
```

5. AWS CLI — Identity & Account

```
# Show which AWS account/user the CLI is currently authenticated as
aws sts get-caller-identity

# Show the same, using a specific named profile
aws sts get-caller-identity --profile sanchay

# List all configured profiles' settings and where each value is sourced from
aws configure list --profile sanchay

# Check whether the account belongs to an AWS Organization
aws organizations describe-organization

# Show the friendly alias set on the current AWS account
aws iam list-account-aliases
```

6. AWS CLI — Credentials & Profiles

```
# Interactively set up a new named credentials profile
aws configure --profile sanchay

# (Bash) Set the active profile for the current terminal session
export AWS_PROFILE=sanchay

# (PowerShell) Set the active profile for the current terminal session
$env:AWS_PROFILE = "sanchay"

# (PowerShell) Make a profile persist across all new terminals
[Environment]::SetEnvironmentVariable("AWS_PROFILE", "sanchay", "User")

# (Bash) Make a profile persist across all new terminals
echo 'export AWS_PROFILE=sanchay' >> ~/.bashrc && source ~/.bashrc

# (Bash) Revert to the default profile for one command
unset AWS_PROFILE

# View raw stored credentials/profiles (Bash)
cat ~/.aws/credentials

# View raw stored credentials/profiles (PowerShell)
type $env:USERPROFILE\.aws\credentials
```

7. AWS CLI — Networking (VPC / Subnets / IP)

```
# List VPCs, their CIDR blocks, and whether each is the account default
aws ec2 describe-vpcs --region us-east-2 \
--query "Vpcs[*].{ID:VpcId,CIDR:CidrBlock,IsDefault:IsDefault,Tags:Tags}"

# List subnets in a given VPC
```

```

aws ec2 describe-subnets --region us-east-2 \
--filters "Name=vpc-id,Values=<vpc-id>" \
--query "Subnets[*].{ID:SubnetId,CIDR:CidrBlock,AZ:AvailabilityZone}"

# List internet gateways attached to a VPC
aws ec2 describe-internet-gateways --region us-east-2 \
--filters "Name=attachment.vpc-id,Values=<vpc-id>" \
--query "InternetGateways[*].InternetGatewayId"

# List non-default security groups in a VPC
aws ec2 describe-security-groups --region us-east-2 \
--filters "Name=vpc-id,Values=<vpc-id>" \
--query "SecurityGroups[*].{ID:GroupId,Name:GroupName}"

# List route tables in a VPC
aws ec2 describe-route-tables --region us-east-2 \
--filters "Name=vpc-id,Values=<vpc-id>" \
--query "RouteTables[*].{ID:RouteTableId,Main:Associations[0].Main}"

# Get the public IP of a specific EC2 instance
aws ec2 describe-instances --region us-east-2 \
--query "Reservations[*].Instances[*].{ID:InstanceId,PublicIP:PublicIpAddress}"

# Get the public IP tied to a network interface (e.g. an ECS Fargate task's ENI)
aws ec2 describe-network-interfaces --network-interface-ids <eni-id> --region us-east-2 \
--query "NetworkInterfaces[*].Association.PublicIp"

# Get your own machine/session's outbound public IP
curl -s https://checkip.amazonaws.com

```

8. AWS CLI — ECS (Fargate)

```

# List ECS clusters in a region
aws ecs list-clusters --region us-east-2

# List services running in a cluster
aws ecs list-services --cluster <cluster-name> --region us-east-2

# List running task ARNs in a cluster
aws ecs list-tasks --cluster <cluster-name> --region us-east-2

# Get task details, including its network interface attachment
aws ecs describe-tasks --cluster <cluster-name> --tasks <task-arn> --region us-east-2 \
--query "tasks[*].attachments[*].details"

# Scale a service down to zero running tasks (before deleting it)
aws ecs update-service --cluster <cluster-name> --service <service-name> --desired-count 0 --region us-east-2

# Delete an ECS service
aws ecs delete-service --cluster <cluster-name> --service <service-name> --region us-east-2

# Delete an ECS cluster
aws ecs delete-cluster --cluster <cluster-name> --region us-east-2

```

9. AWS CLI — Application Load Balancer

```

# List load balancers and their ARNs/DNS names
aws elbv2 describe-load-balancers --region us-east-2 \

```

```
--query "LoadBalancers[*].{Name:LoadBalancerName,ARN:LoadBalancerArn}"

# List target groups and their ARNs
aws elbv2 describe-target-groups --region us-east-2 \
--query "TargetGroups[*].{Name:TargetGroupName,ARN:TargetGroupArn}"

# Delete a load balancer
aws elbv2 delete-load-balancer --load-balancer-arn <arn> --region us-east-2

# Delete a target group
aws elbv2 delete-target-group --target-group-arn <arn> --region us-east-2
```

10. AWS CLI — ECR (Elastic Container Registry)

```
# List ECR repositories in a region
aws ecr describe-repositories --region us-east-2

# List images (tags, digests, push dates) in a repository
aws ecr describe-images --repository-name sanchay-api --region us-east-2

# Delete an ECR repository (and any images in it)
aws ecr delete-repository --repository-name sanchay-dev-api --region us-east-2 --force

# Authenticate Docker to ECR
aws ecr get-login-password --region us-east-2 | docker login --username AWS --password-stdin
263011180772.dkr.ecr.us-east-2.amazonaws.com
```

11. Docker — Build & Push

```
# Build a local Docker image
docker build -t sanchay-api .

# Tag a local image for an ECR repository
docker tag sanchay-api:latest 263011180772.dkr.ecr.us-east-2.amazonaws.com/sanchay-api:latest

# Push the tagged image to ECR
docker push 263011180772.dkr.ecr.us-east-2.amazonaws.com/sanchay-api:latest

# Check installed Docker version
docker --version
```

12. AWS CLI — SSM Parameter Store

```
# Fetch and decrypt a stored secret value into a shell variable
DATABASE_URL=$(aws ssm get-parameter --name /sanchay-api/database-url --with-decryption \
--region us-east-2 --query "Parameter.Value" --output text)
```

13. AWS Secrets Manager

```
# Create a secret (e.g. GHCR credentials for ECS to pull a private image)
aws secretsmanager create-secret \
--name sanchay/dev/ghcr-credentials \
--secret-string '{"username":"chandramcsr","password":"<PAT>"}' \
--region us-east-2
```

14. GitHub Actions / GitHub CLI

```
# View a workflow file's contents (Bash)
cat .github/workflows/docker.yml

# Trigger a workflow manually (needs workflow_dispatch in the 'on:' block)
gh workflow run docker.yml --repo chandramcsr/sanchay-api

# Trigger a workflow with no code change, via an empty commit
git commit --allow-empty -m "Trigger build" && git push
```

15. Git

```
# Clone a repository
git clone https://github.com/chandramcsr/sanchay-infrastructure.git
```

16. Terraform Install / Upgrade (Windows)

```
# Install or upgrade Terraform via winget
winget install HashiCorp.Terraform

# Upgrade an existing winget install
winget upgrade HashiCorp.Terraform

# Install/upgrade via Chocolatey
choco upgrade terraform

# Find where the terraform.exe binary currently lives on PATH
where.exe terraform
```